Project 19 (Real-World Example) — RepoSage-Codebase-Q-And-A-Assistant-With-File-Citations

Type: RAG Q&A over code · **Stack:** Python, LangChain, Chroma (RAG), `langchain-text-splitters`, OpenAI SDK · **Level:** Intermediate

Tagline: Ask a question about a codebase — "where is auth handled?", "what does `connect()` do?" — and get an answer grounded in the actual source, with the **files it came from** cited.

Charter (what this project is)

Dropping into an unfamiliar repo, the first hour is spent grepping and guessing where things live. **RepoSage** answers questions about a codebase from the code itself: it indexes the repo's source files with RAG, retrieves the snippets most relevant to your question, and has the model answer **only from those snippets** — citing the files they came from, so you can jump straight to the source. It's DocuMind's RAG (Project 3) pointed at code instead of prose, where the citation is a file path you can open.

Jump to: Project Structure · Setup · How testing works · Step 1 · Step 2 · Step 3

The Story — what you're building

A question comes in. The repo's source files are chunked, embedded, and stored in Chroma, with each chunk remembering **which file** it came from. A **retriever** pulls the few snippets closest to the question. A deterministic **context builder** stitches those snippets into one block with a `# File: path` header on each, and collects the **unique file list** for the citations. The **answer** step asks the model to answer *only* from that context, and the result is printed with the cited files.

The lesson: a codebase assistant is RAG with one twist — the "documents" are code, and the **citation is a file path**. The context builder and the citation list are deterministic (snippets in, formatted context + file list out), so you test them with hand-made snippets, no model or embeddings. The model only turns the retrieved code into an answer.

Project Structure (the professional layout)

`main.py` takes a question; all code in `src/`.

```
<repo folder>/ (the RepoSage repo — full name in the heading above)
├── sample_repo/                a tiny codebase to index (a few .py files)
├── src/
│   ├── models/
│   │   ├── __init__.py
│   │   └── llm.py
│   ├── rag/
│   │   ├── __init__.py
│   │   ├── code_store.py       build_code_store()        (Step 1)
│   │   └── retriever.py        retrieve_snippets()       (Step 1)
│   ├── utils/
│   │   ├── __init__.py
│   │   └── logger.py
│   ├── __init__.py
│   ├── answer.py               answer_question()         (Step 2)
│   ├── config.py
│   ├── context.py              build_context(), cited_files() (Step 1)
│   ├── exceptions.py
│   └── pipeline.py             run_query()               (Step 3)
├── tests/
│   ├── integration/test_live_query.py
│   └── unit/                   test_config, test_context
├── .env.example
├── .gitignore
├── LICENSE
├── main.py
├── README.md
└── requirements.txt
```

How the structure grows, step by step

- **Step 1** builds the standard setup, `src/rag/` (code store + retriever) and `src/context.py` (the deterministic context + citation builder) with tests — the builders tested with no model.
- **Step 2** adds `src/answer.py` (the grounded answer chain).
- **Step 3** adds `src/pipeline.py`, `main.py`, and the integration test.

Setup (do this once, before Step 1)

```
cd RepoSage-Codebase-Q-And-A-Assistant-With-File-Citations
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install langchain-openai langchain-community langchain-text-splitters \
```

```
    chromadb openai python-dotenv
pip freeze > requirements.txt
```

`.env.example` (copy to `.env`): `OPENAI_API_KEY=sk-...`, `LOG_LEVEL=INFO`. `.gitignore`: `.env`, `.venv/`, `__pycache__/`, `*.pyc`, `chroma_db/`. Every folder under `src/` gets an empty `__init__.py`. Put a tiny codebase in `sample_repo/` — 3-4 `.py` files with clearly different jobs (e.g. `auth.py`, `db.py`, `api.py`) — so questions have distinct files to find and cite. `LICENSE` is the MIT text.

How testing works in this project

Two layers, one style (plain OK/WRONG scripts; this builds on Doc08, not pytest):

- **`tests/unit/`** — no model, no embeddings: `test_config`; `test_context` (the context builder + citation list, with hand-made snippets — every snippet gets a `# File:` header, and the cited-files list is de-duplicated and in first-seen order).
- **`tests/integration/test_live_query.py`** — real model + RAG: a question about the sample repo is answered from the code and names the right file(s).

Each step ends with a **Tests for this step** box.

Step 1 — The Code Index and the Context Builder

What this step does: the code store, the retriever, and the deterministic `build_context` / `cited_files` — the parts that turn retrieved snippets into a model-ready block and a citation list, with no model.

Why this step matters: the retrieval and formatting are what ground the answer; proving the context builder and citations with hand-made snippets means Step 2's model call is the only unproven piece left.

Stuck? Hint 1 · Hint 2 · Solution

What to do: 1. Write the standard setup (`config` / `exceptions` / `utils/logger` / `models/llm`). 2. Write `src/rag/code_store.py`: `build_code_store(repo_dir)` — load `**/*.py`, chunk (`RecursiveCharacterTextSplitter`), keep each chunk's file path in `metadata["file"]`, embed, add to Chroma. `src/rag/retriever.py`: `retrieve_snippets(collection, question, k=4)` → a list of `{"code": ..., "file": ...}`. 3. Write `src/context.py`: `build_context(snippets)` — join each snippet under a `# File: <path>` header; `cited_files(snippets)` — the unique file paths, in first-seen order.

Tests for this step: `test_context` — every snippet has a `# File:` header in the context; `cited_files` de-duplicates (two snippets from `auth.py` → `auth.py` once). `test_config`.

Step 2 — The Grounded Answer

What this step does: `answer_question(model, question, context)` — ask the model to answer the question using **only** the retrieved code context.

Why this step matters: grounding is the whole point — the answer must come from the repo's actual code, not the model's training memory, so it's correct for *this* codebase.

What's new vs. Step 1: `src/answer.py`. **What stays the same:** the index and the context builder.

Stuck? Hint 1 · Hint 2 · Solution

What to do: 1. Write `src/answer.py`: an `ANSWER_PROMPT` that says "answer using only the code below; if it's not there, say you don't know", and `answer_question(model, question, context)` = `(prompt | model).invoke(...)` returning the text.

Tests for this step: proven live in Step 3; for now, `answer_question` returns a string given a hand-written context.

Step 3 — The Pipeline (final)

What this step does: `run_query` ties it together — index the repo, retrieve snippets, build the context, answer, and attach the cited files — plus `main.py` and the integration test.

Why this step matters: this is the one call a user makes: a question in, a grounded answer plus the files it came from out.

What's new vs. Step 2: `src/pipeline.py`, `main.py`. **What stays the same:** everything else.

Stuck? Hint 1 · Hint 2 · Solution

What to do: 1. Write `src/pipeline.py`: `run_query(model, repo_dir, question)` — `build_code_store` → `retrieve_snippets` → `build_context` → `answer_question`, returning the answer and `cited_files(snippets)`. 2. Write `main.py` (take a question, print the answer and the cited files) and `tests/integration/test_live_query.py`.

Final Deliverable: an assistant that answers questions about a codebase from the code, citing the files the answer came from.

Tests for this step: `test_live_query` — a question about the sample repo is answered from the code and cites the relevant file.

Run, Test and Configure the Final Project

```
python main.py "How is login handled?"
python -m tests.unit.test_config
python -m tests.unit.test_context
python -m tests.integration.test_live_query
```

Variable	Required	Default	Meaning
`OPENAI_API_KEY`	yes	—	Your OpenAI API key
`LOG_LEVEL`	no	`INFO`	Log verbosity
`REPO_DIR`	no	`sample_repo`	The codebase to index

Checklist Before You Call This Done

- [ ] Each code chunk remembers its file path (the citation)
- [ ] The answer is grounded in retrieved code, not the model's memory
- [ ] The context builder and citation list are deterministic, tested with no model
- [ ] The answer names the file(s) it came from
- [ ] An off-topic question gets "I don't know", not an invented answer

Publish It to GitHub

Name the repo `RepoSage-Codebase-Q-And-A-Assistant-With-File-Citations`, add the `README.md`, push. "Ask my codebase a question" is a demo every engineer immediately wants.

Status

Not started.

Stuck? Ask for **Hint 1** or **Hint 2** before the solution. Build the context + citation builders (Step 1) first — they're deterministic and need no model.